

Unit Tests

Der größte Feind eines Developers



The only good bug is a dead bug!

"It's not a bug, it's a feature!"

Siehe auch: [https://en.wikipedia.org/wiki/Bug_\(engineering\)](https://en.wikipedia.org/wiki/Bug_(engineering))

6 Stages of Debugging

1. That can't happen.
2. That doesn't happen on my machine.
3. That shouldn't happen.
4. Why does that happen?
5. Oh, I see. 
6. How did that ever work?!

Debugging

Verwende nicht `IO.println()`, um nach Fehlern zu suchen.

Nutze stattdessen Breakpoints und die Debug-Funktion in der IDE.

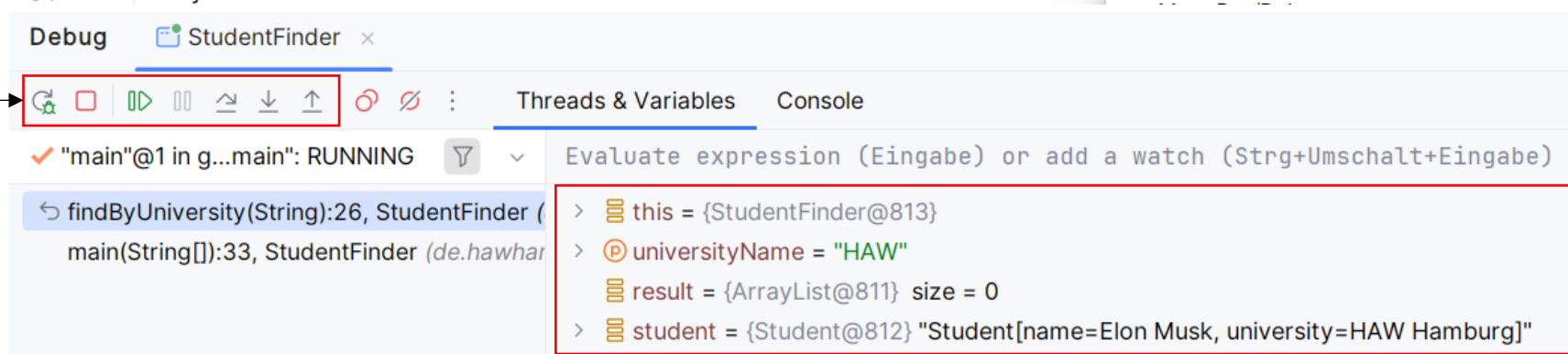
Setze Breakpoint →



```
20 List<Student> findByUniversity(String universityName) { 2 usages
21     universityName = universityName.trim();
22
23     List<Student> result = new ArrayList<>();
24     for (Student student : students) {
25         if (student.university().contains(universityName)) {
26             result.add(student);
27         }
28     }
29     return result;
30 }
31
32 public static void main(String[] args) {
33     System.out.println(new StudentFinder().findByUniversity(universityName));
34 }
```

Befehle:

- Rerun
- Stop
- Resume
- Step over
- Step into



Debug StudentFinder x

Threads & Variables Console

✓ "main"@1 in g...main": RUNNING

findByUniversity(String):26, StudentFinder (de.hawhar)

main(String[]):33, StudentFinder (de.hawhar)

Evaluate expression (Eingabe) or add a watch (Strg+Umschalt+Eingabe)

- > this = {StudentFinder@813}
- > universityName = "HAW"
- > result = {ArrayList@811} size = 0
- > student = {Student@812} "Student[name=Elon Musk, university=HAW Hamburg]"

Starte Debugger

Aktuelle Werte
in den Variablen

Debugging

Debugging erlaubt es, ein Programm zur Laufzeit zu analysieren, z. B. den Ablauf prüfen oder welche konkreten Werte vorliegen.

Allerdings ist Debugging ein **manueller Prozess**, der langwierig sein kann und Konzentration erfordert.

Wenn es doch nur eine automatisierte Lösung gäbe...





JUnit Test

JUnit Test ist ein Programm (Framework), das **automatisiert testet**, ob ein anderes Programm tut, was es soll.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
public class CalculatorTest {
    @Test
    void sumOfTwoNumbersCanBeCorrectlyCalculated() {
        // Arrange
        Calculator calculator = new Calculator();
        int numberA = 1;
        int numberB = 1;

        // Act
        int actualResult = calculator.add(numberA, numberB);

        // Assert
        assertEquals(2, actualResult);
    }
}
```

Testklassen heißen häufig wie die zu testende Klasse mit einem Test-Suffix.

Die Annotation `@Test` markiert die nachfolgende Methode als Testfall.

Assert-Methoden sind spezielle Methoden aus der JUnit-Bibliothek.

JUnit Test – Alles ist gut

Alles ist in Ordnung.
Schlaf gut.

```
3  import org.junit.jupiter.api.Test;
4
5  import static org.junit.jupiter.api.Assertions.assertEquals;
6
7  public class CalculatorTest {
8
9      @Test
10     void sumOfTwoNumbersCanBeCorrectlyCalculated() {
11         // Arrange
12         Calculator calculator = new Calculator();
13         int numberA = 1;
14         int numberB = 1;
15
16         // Act
17         int actualResult = calculator.add(numberA, numberB);
18
19         // Assert
20         assertEquals(expected: 2, actualResult);
21     }
22 }
```

```
3  import org.junit.jupiter.api.Test;
4
5  import static org.junit.jupiter.api.Assertions.assertEquals;
6
7  public class CalculatorTest {
8
9      @Test
10     void sumOfTwoNumbersCanBeCorrectlyCalculated() {
11         // Arrange
12         Calculator calculator = new Calculator();
13         int numberA = 1;
14         int numberB = 1;
15
16         // Act
17         int actualResult = calculator.add(numberA, numberB);
18
19         // Assert
20         assertEquals(expected: 2, actualResult);
21     }
22 }
```

JUnit Test – BUG!!!

FU**!

```
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6
7 public class CalculatorTest {
8
9     @Test
10     void sumOfTwoNumbersCanBeCorrectlyCalculated() {
11         // Arrange
12         Calculator calculator = new Calculator();
13         int numberA = 1;
14         int numberB = 1;
15
16         // Act
17         int actualResult = calculator.add(numberA, numberB);
18
19         // Assert
20         assertEquals(expected: 2, actualResult);
21     }
22 }
```

```
class Calculator {
    public int add(int numberA, int numberB) {
        return 3;
    }
}
```


JUnit Test – Bugfix



Alles ist in Ordnung.
Schlaf gut.

```
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6
7 public class CalculatorTest {
8
9     @Test
10     void sumOfTwoNumbersCanBeCorrectlyCalculated() {
11         // Arrange
12         Calculator calculator = new Calculator();
13         int numberA = 1;
14         int numberB = 1;
15
16         // Act
17         int actualResult = calculator.add(numberA, numberB);
18
19         // Assert
20         assertEquals(expected: 2, actualResult);
21     }
22 }
```

```
class Calculator {
    public int add(int numberA, int numberB) {
        return 2;
    }
}
```

JUnit Test – Echter Bugfix

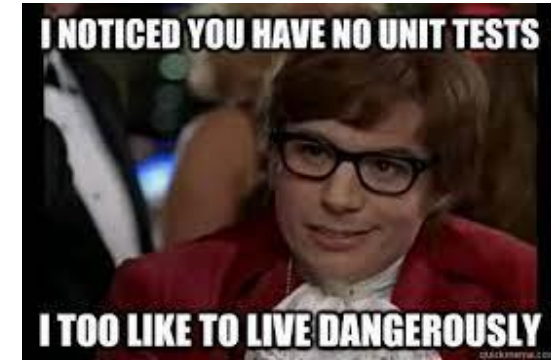


Alles ist in Ordnung.
Schlaf gut.

```
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6
7 public class CalculatorTest {
8
9     @Test
10     void sumOfTwoNumbersCanBeCorrectlyCalculated() {
11         // Arrange
12         Calculator calculator = new Calculator();
13         int numberA = 1;
14         int numberB = 1;
15
16         // Act
17         int actualResult = calculator.add(numberA, numberB);
18
19         // Assert
20         assertEquals(expected: 2, actualResult);
21     }
22 }
```

```
class Calculator {
    public int add(int numberA, int numberB) {
        return numberA + numberB;
    }
}
```





Warum Testen? 🤔

- Beweisen, dass es funktioniert.
- Schnelles Feedback erhalten.
- Refactoring ohne Angst (siehe Folien "Refactoring").
- Sicherstellen, dass die Spezifikation eingehalten wird.
z. B. Performance, Datendurchsatz, Sicherheit.
- Anforderungen klären.
- Design Feedback: Ist der Code leicht testbar? Ansonsten Design-Problem?
- Für die Testabdeckung, da Prozent-Vorgabe vom Management existiert...
- [Software Bug Iceberg \(Youtube\)](#)

Qualitäten eines Tests

- Sensibel: erkennt Änderungen im Verhalten.
- Funktional: nicht an die Implementierung gekoppelt.
D. h. widerstandsfähig gegenüber Refactoring. #FragileTests
- Spezifisch: versagt aus *einem* eindeutigen Grund.
- Ausdrucksstark: verwendet Fachbegriffe.
- Automatisiert: ohne bzw. mit minimalen manuellen Eingriffen ausführbar.
- Wiederholbar: liefert bei erneuter Ausführung das gleiche Ergebnis.
- Isoliert: gegenüber anderen Tests / externem Systemzustand.
- Minimal: enthält nur relevante Dinge.
- Prinzipien des Refactorings gelten auch für das Schreiben von Unit Tests.



FIRST-Regeln

Fast: Tests sollten zügig ablaufen. Wenn Tests langsam laufen, werden sie nicht häufig durchgeführt. Wenn Tests nicht häufig durchgeführt werden, werden Probleme nicht früh genug gefunden. Wenn Probleme nicht früh genug gefunden werden, sind sie schwerer zu beheben. Der Code wird anfangen zu rotten.

Independent: Tests sollten nicht voneinander abhängig sein. Jeder Test sollte unabhängig und in beliebiger Reihenfolge durchführbar sein. Andernfalls führt ein fehlgeschlagener Test zu einer Kaskade von Folgefehlern, was die Diagnose erschwert.

Repeatable: Tests sollten in jeder Umgebung wiederholbar sein. Tests sollen in der Produktionsumgebung, in der QA-Umgebung und auf deinem Laptop auf der Heimfahrt im Zug ohne Netzwerk durchführbar sein. Andernfalls führt dies nur zu Ausreden, warum Tests fehlschlagen (was Tests nutzlos macht).

Self-Validating: Tests sollten entweder bestehen oder fehlschlagen. Man soll nicht erst eine Protokolldatei lesen müssen, um festzustellen, ob ein Test bestanden wurde. Wenn ein Test nicht selbstüberprüfend ist, wird das Scheitern des Tests subjektiv und erfordert eine lange, manuelle Auswertung.

Timely: Tests müssen zeitnah geschrieben werden. Tests sollten unmittelbar vor dem Produktionscode geschrieben werden. Tests, die im Nachhinein geschrieben werden, führen dazu, dass Produktionscode nicht so gestaltet wird, dass er testbar ist.

Testarten



Es gibt unterschiedliche Arten, um Software zu testen. Zum Beispiel:

- **Unit Test:** Testet *eine* Klasse oder *ein* Konzept isoliert.
- **Service/Integration Test:** Testet *mehrere* Klassen oder Konzepte gleichzeitig.
z. B. Zusammenspiel mehrerer Klassen mit einer Datenbank oder API.
- **UI Test:** Testet die Benutzeroberfläche selbst.
- **End-to-End Test:** Testet die komplette Anwendung über die Benutzeroberfläche bzw. API (Schnittstelle).
- Weitere...

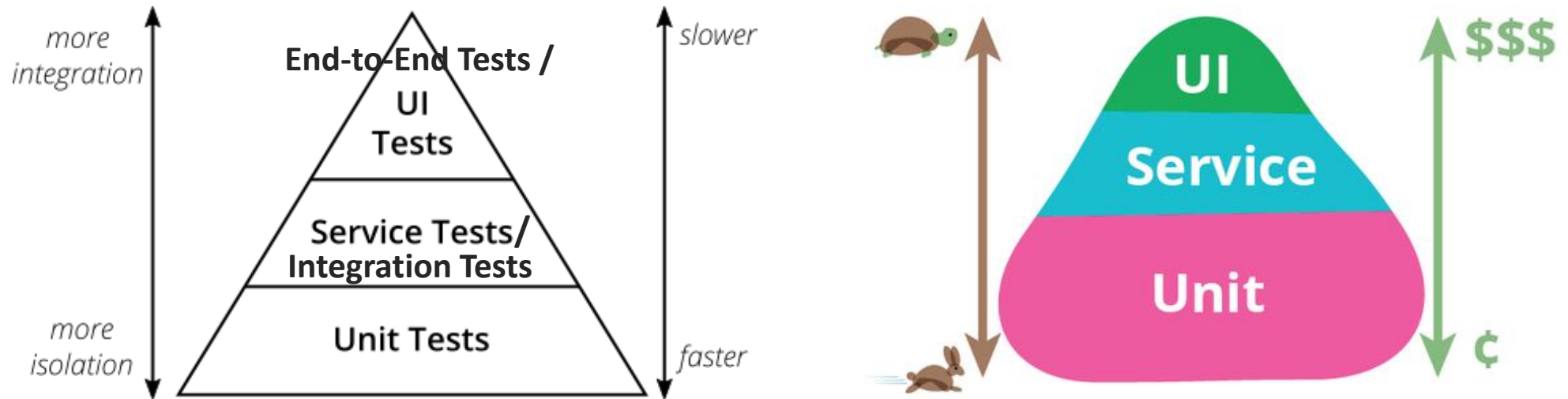
Teststrategie

Eine Teststrategie ist ein Plan, der die Ziele, Methoden, Ressourcen und Zeitrahmen für das Testen einer Software definiert. Sie dient als Leitfaden für das gesamte Testteam und stellt sicher, dass die Tests effektiv und effizient durchgeführt werden.

Eine der bekanntesten Strategien ist die **Testpyramide**.



Testpyramide



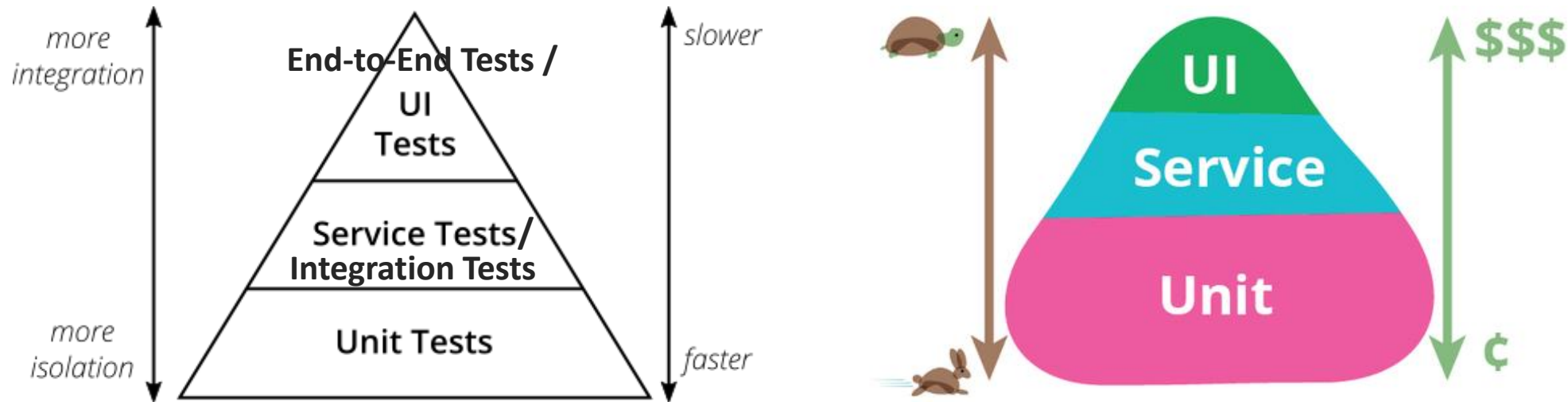
"Stick to the pyramid shape to come up with a healthy, fast and maintainable test suite: Write *lots* of small and fast *unit tests*. Write *some* more coarse-grained tests and *very few* high-level tests that test your application from end to end. Watch out that you don't end up with a test ice-cream cone that will be a nightmare to maintain and takes way too long to run."

Quellen:

<https://martinfowler.com/articles/practical-test-pyramid.html>

<https://martinfowler.com/bliki/TestPyramid.html>

Testpyramide



Achtung:

Unit Test hat eine doppelte Bedeutung, die vom Kontext abhängt:

- **Unit Test als Testart:** meint *eine* Klasse isoliert zu testen ohne ihre Abhängigkeiten. Das heißt, dass sämtliche Programmlogik der abhängigen Klassen für den Unit Test durch Konstanten ersetzt wird.
- **Unit Test als Framework:** wird verwendet, um verschiedene Testarten umzusetzen und automatisch auszuführen. Die verschiedenen Testarten in der Testpyramide (siehe oben) würde man z. B. alle mit JUnit schreiben und testen.

Test-Driven Development (TDD)

„Test First“

Schreibe erst den Testfall, dann den zugehörigen Programm-Code.

Vorteile:

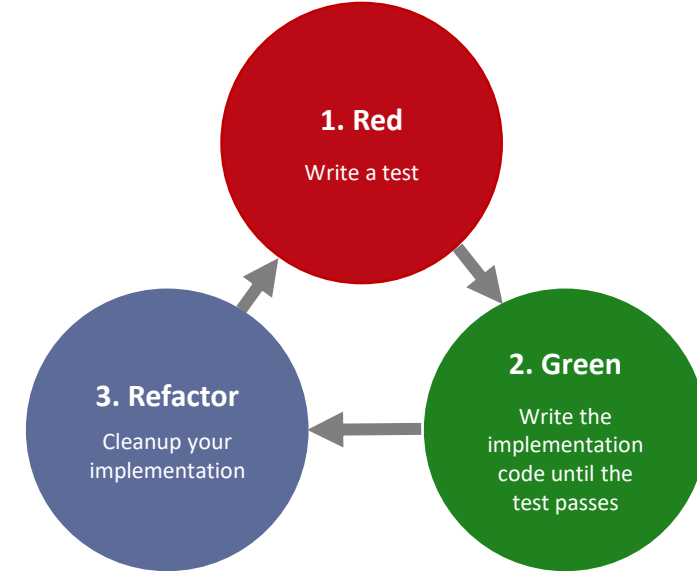
- Man macht sich vorab strukturiert Gedanken, welche Rückgabewerte und Parameter ein Feature benötigt, um es besser testen zu können.
- Die Anforderungen für das Feature lassen sich über die Tests vorab definieren.

Nachteile:

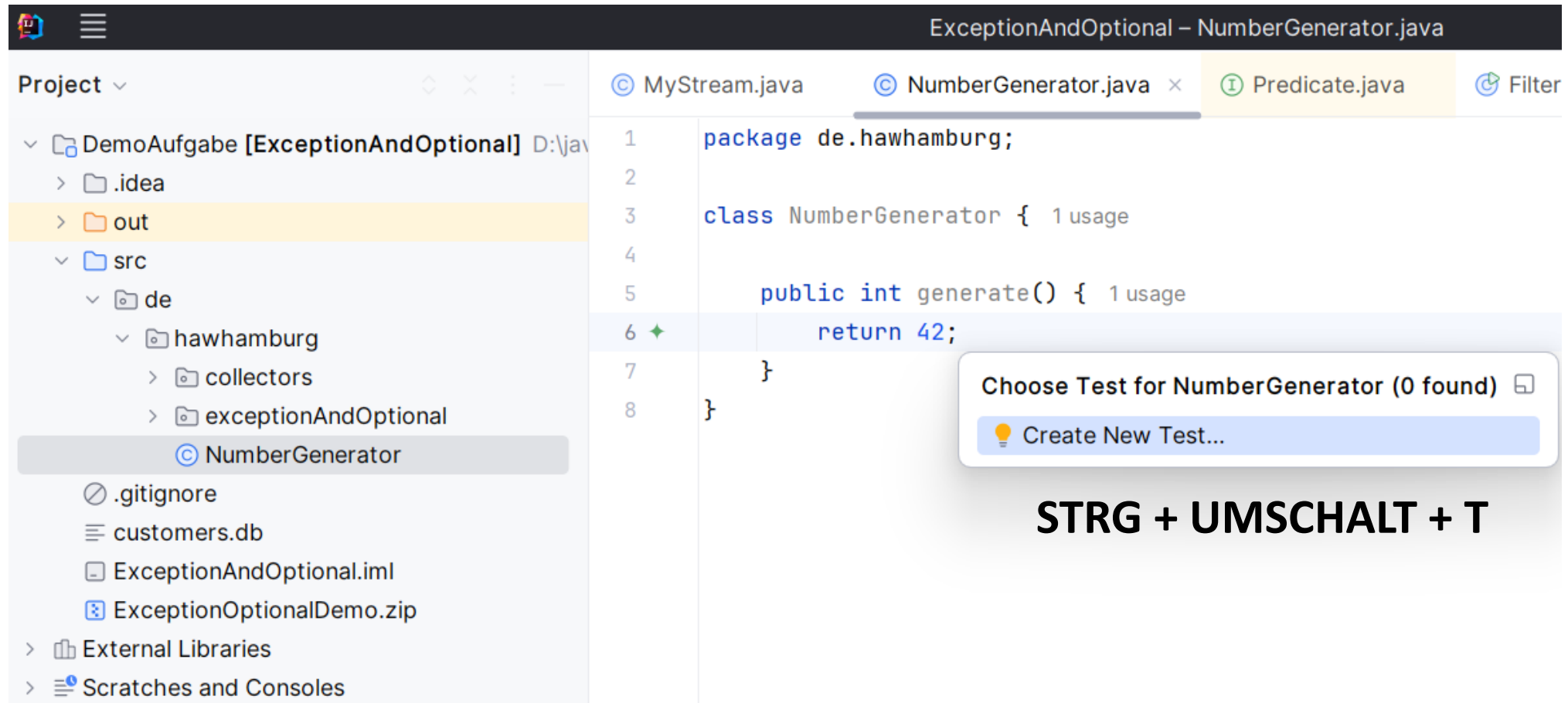
- Führt unter Umständen zu kleinteiligen Schritten und Refactorings, die übersprungen werden können.

“According to our findings, the benefits of test-driven development compared to iterative test-last development are small and thus in practice relatively unimportant, although effects are positive. There is an indication of test-driven development endorsing better branch coverage, but effect size is considered small.”

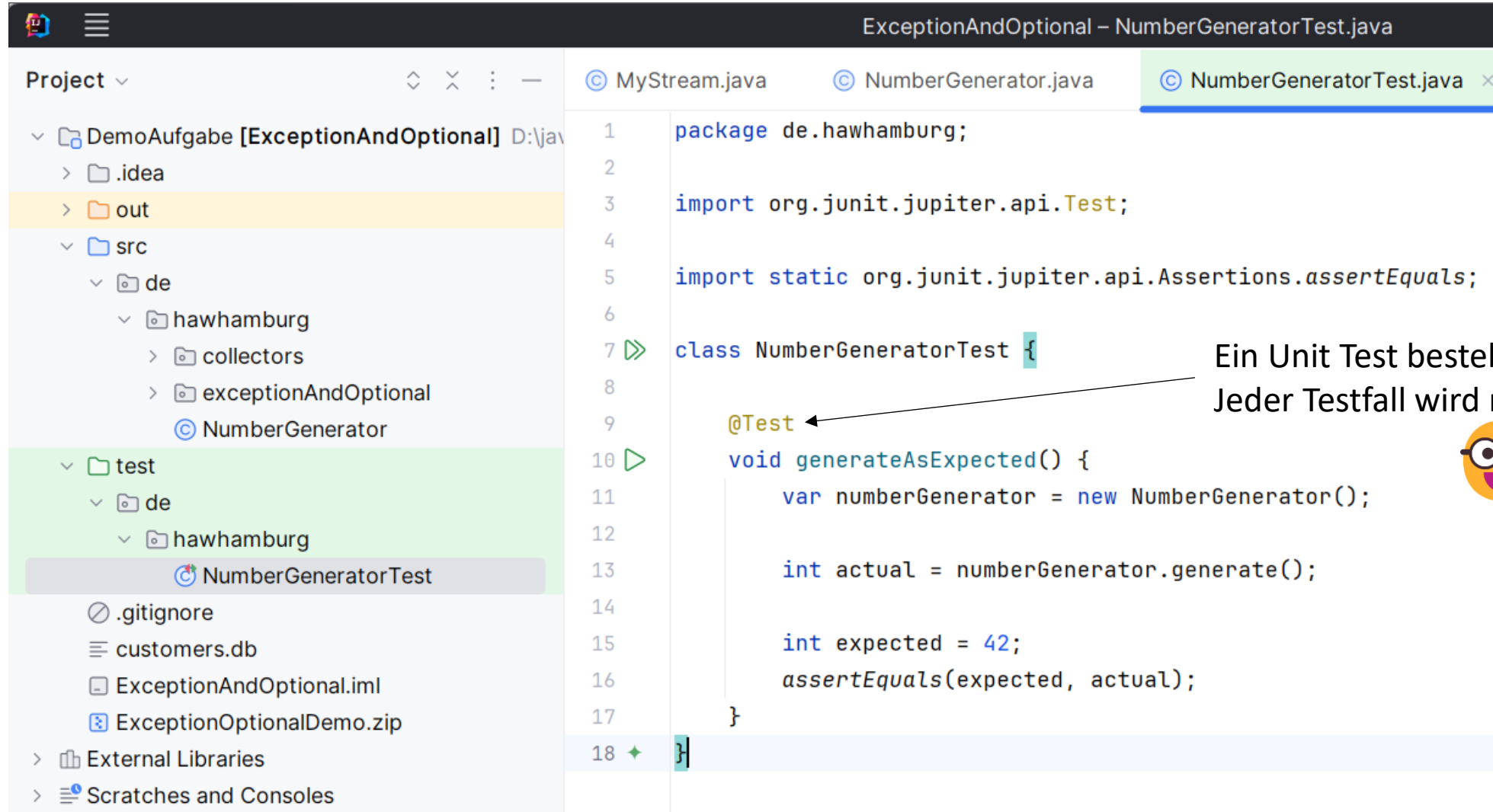
Quelle: <https://www.sciencedirect.com/science/article/abs/pii/S0950584911000346>



Unit Test mit IntelliJ erstellen



Unit Test in IntelliJ



ExceptionAndOptional – NumberGeneratorTest.java

Project ▾

- ▾ DemoAufgabe [ExceptionAndOptional] D:\jav
 - > .idea
 - > out
 - ▾ src
 - ▾ de
 - ▾ hawhamburg
 - > collectors
 - > exceptionAndOptional
 - © NumberGenerator
 - ▾ test
 - ▾ de
 - ▾ hawhamburg
 - © NumberGeneratorTest
 - .gitignore
 - customers.db
 - ExceptionAndOptional.iml
 - ExceptionOptionalDemo.zip
 - > External Libraries
 - > Scratches and Consoles

```
1 package de.hawhamburg;  
2  
3 import org.junit.jupiter.api.Test;  
4  
5 import static org.junit.jupiter.api.Assertions.assertEquals;  
6  
7 class NumberGeneratorTest {  
8  
9     @Test  
10    void generateAsExpected() {  
11        var numberGenerator = new NumberGenerator();  
12  
13        int actual = numberGenerator.generate();  
14  
15        int expected = 42;  
16        assertEquals(expected, actual);  
17    }  
18 }
```

Ein Unit Test besteht aus Testfällen.
Jeder Testfall wird mit `@Test` annotiert.



Übung: Mein erster JUnit Test



Schreibe einen Unit Test für folgende Klasse:

```
import java.util.Set;
import java.util.stream.Collectors;

class SetUtil {

    /**
     * Berechnet die Schnittmenge der beiden Mengen.
     */
    public static Set<Integer> intersection(Set<Integer> set1, Set<Integer> set2) {
        return set1.stream()
            .filter(e -> set2.contains(e))
            .collect(Collectors.toSet());
    }
}
```

JUnit 5 – Asserts und Annotations

Assert-Methoden:

z. B. *assertEquals*, *assertTrue*, *assertFalse*, *assertNull*, *assertNotNull*, *assertThrows*, *assertAll*, *assertIterableEquals*, *assertArrayEquals*, *assertLinesMatch*, *assertSame*, *assertNotSame*, *assertTimeout*, *fail*

Annotationen:

z. B. *@BeforeAll*, *@BeforeEach*, *@Test*, *@AfterEach*, *@AfterAll*, *@RepeatedTest(int)*

Exkurs: Was sind static imports??

```
package utils;
```

```
class NumberUtil {  
1.   public static boolean isNumber(Object object) {  
       return object instanceof Byte ||  
              object instanceof Short ||  
              object instanceof Integer ||  
              object instanceof Long ||  
              object instanceof Float ||  
              object instanceof Double;  
   }  
}
```

```
2.   import static utils.NumberUtil.isNumber;
```

```
class Demo {  
   public static void main(String[] args) {  
3.       if (isNumber(42)) {  
           System.out.println("42 is a number.");  
       }  
   }  
}
```

Eine statische Methode kann verwendet werden, als wäre sie Teil der Klasse.

Statisch importierte Methoden werden häufig in Unit Tests eingesetzt wie z. B. *assertEquals*. Die folgende Zeile importiert alle statischen Methoden, die in der Klasse [Assertions](#) enthalten sind:

```
import static org.junit.jupiter.api.Assertions.*;
```

JUnit 5 – Achte auf deine Imports! 😏

JUnit 4

```
import static org.junit.Assert.*;
```

```
@Test from org.junit
```

```
@RunWith
```

```
@Before / @BeforeClass
```

```
@After / @AfterClass
```

```
@Ignore
```

```
@Category
```

```
@Rule / @ClassRule
```

JUnit 5

```
import static org.junit.jupiter.api.Assertions.*;
```

```
@Test from org.junit.jupiter.api.test
```

```
@ExtendWith / @RegisterExtension
```

```
@BeforeEach / @BeforeAll
```

```
@AfterEach / @AfterAll
```

```
@Disabled
```

```
@Tag
```

😱 #doomsday wenn du JUnit 4 und JUnit 5 miteinander vermischt.

Verhalten vs Implementation



Ein Unit Test prüft das **Verhalten**,
nicht die **Implementation**.

Verhalten vs Implementation: Beispiel 1/2

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class StringUtilTest {

    @Test
    void countCharacters() { ←
        int actual = StringUtil.countCharacters("Elona");

        assertEquals(4, actual);
    }
}
```

Der Testfall interessiert sich nur für das **Verhalten** (hier das Ergebnis).

Hinweis: Das Verhalten ist abhängig von den Parametern (hier der übergebene String).

Dem Testfall ist egal, wie das Ergebnis zustande kommt (wie es implementiert wurde).

Verhalten vs Implementation: Beispiel 2/2

```
final class StringUtil {  
  
    static int countCharacters(String text) {  
        return text.length();  
    }  
  
    static int countCharacters(String text) {  
        int result = 0;  
        for (byte _ : text.getBytes()) {  
            result++;  
        }  
        return result;  
    }  
  
    static int countCharacters(String text) {  
        if(text.isEmpty()) {  
            return 0;  
        }  
        return countCharacters(text.substring(1)) + 1;  
    }  
}
```

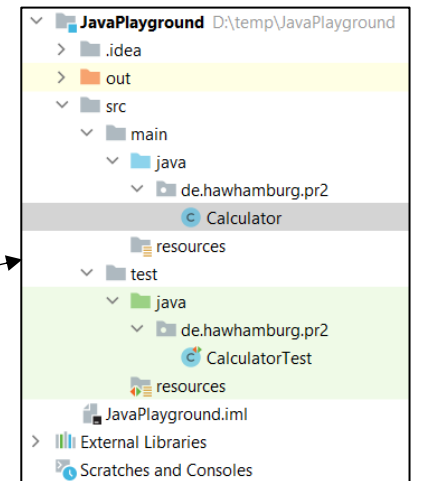
Dies sind drei mögliche Implementationen für die Methode countCharacters.

Welche Implementation verwendet wird, ist dem Unit Test egal, da es nur an dem Verhalten interessiert ist.

Verhalten:
Spezifische Eingabewerte führen zu einem bestimmten Ergebnis.

Best Practices 🧐

1. JUnit: erwarteter Wert kommt zuerst: `assertEquals(expected, actual);` ← Werden die Werte vertauscht, erhält man verwirrende Fehlermeldungen.
 2. Benenne Testfälle möglichst prägnant: was wird getestet und warum?
 3. Nutze das AAA-Pattern.
 4. Testfälle müssen unabhängig von anderen Testfällen sein.
 5. Teste ein Konzept pro Testfall: möglichst ein `assert` pro Methode.
 6. Pushe niemals Code mit fehlgeschlagenen Tests ins Git-Repository.
 7. Vorsicht beim Vergleich von Gleitkommawerten.
 8. Habe eine hohe Testabdeckung inkl. Randfälle (Edge-Cases).
 9. Verwende Äquivalenzklassen.
 10. Verwende die [Standard-Ordnerstruktur](#) von Apache.
- Optional: Verwende Erweiterungen wie AssertJ, Truth, Hamcrest.





Best Practices: Namen für die Testfälle

Für die Benennung der Testfälle wird häufig das Given-When-Then-Schema empfohlen (stammt aus [Behavior-Driven Development](#)), allerdings können die Methodennamen der Testfälle etwas sperrig werden.

Given

When

Then

```
void givenAnInactiveCustomer_whenHePlacesAnOrder_itIsActivated()
```

```
void givenAProgramLinkedTo2Records_whenUpdated_2AuditsAreInserted()
```

Ein alternativer Vorschlag verwendet für jedes Given eine eigene Testklasse:

Testklasse

Testfall

```
AnInactiveCustomerTest::beActivatedWhenCustomerPlacesAnOrder()
```

Siehe auch: <https://www.codurance.com/publications/2014/12/13/naming-test-classes-and-methods>

Warum? Die Methodennamen werden bei fehlgeschlagenen Tests angezeigt.



Best Practices: AAA-Pattern

Innerhalb der Testfälle wird das AAA-Schema empfohlen oder alternativ given-when-then:

Arrange (given)

Initialisiere die benötigten Testdaten.

Act (when)

Führe die zu testende Funktionalität aus.

Assert (then)

Prüfe, ob das Ergebnis bzw. Verhalten mit der Erwartung übereinstimmt.



Best Practices: AAA-Pattern Beispiel

```
class StudentFinderTest {  
  
    @Test  
    void findStudentsByUniversity() {  
        // Arrange (given)  
        List<Student> expected = new ArrayList<>(List.of(  
            new Student("Milla Maxwell", "HAW Hamburg"),  
            new Student("Jude Mathis", "HAW Hamburg")  
        ));  
  
        // Act (when)  
        List<Student> actual = new StudentFinder().findByUniversity("HAW Hamburg");  
  
        // Assert (then)  
        assertEquals(expected, actual);  
    }  
}
```

Best Practices: Unabhängige Testfälle

```
class DatabaseTest {  
    private static DatabaseConnection databaseConnection;  
    private List<String> elements;
```

@BeforeAll

```
static void init() {
```

```
    databaseConnection = new DatabaseConnection("localhost");
```

```
    databaseConnection.connect(); // Stelle einmalig eine Verbindung zur Datenbank für den gesamten Unit Test her.
```

```
}
```

@BeforeEach

```
void setUp() {
```

```
    // Initialisiere elements nach jedem Testfall neu (falls elements in irgendeinen Testfall verändert wird, wird dies hier wieder rückgängig gemacht).
```

```
    elements = new ArrayList<>();
```

```
    elements.add(databaseConnection.getElement(0)); // returns "Element 1"
```

```
    elements.add(databaseConnection.getElement(1)); // returns "Element 2"
```

```
}
```

@Test

```
void someTest1() {
```

```
    elements.removeFirst();
```

```
    System.out.println(elements);
```

```
}
```

@Test

```
void someTest2() {
```

```
    elements.removeFirst();
```

```
    System.out.println(elements);
```

```
}
```

@AfterAll

```
static void tearDown() {
```

```
    databaseConnection.disconnect(); // Schließe einmalig die Verbindung nach Ausführung des gesamten Unit Tests.
```

```
}
```

```
}
```

Die Methode **@BeforeAll static void init()** wird **einmalig vor allen Testfällen** ausgeführt.

Die Methode **@BeforeEach void setUp()** wird vor **jedem Testfall** ausgeführt.

Jeder **Testfall** soll **unabhängig** von allen anderen Testfällen sein. D. h. ein Testfall darf **keine Seiteneffekte** haben! Hätte ein Testfall Seiteneffekte auf andere Testfälle, dann würde die Ausführungsreihenfolge der Testfälle eine Rolle spielen. Aus dem Grund werden **Testfälle von JUnit in zufälliger Reihenfolge ausgeführt**: eine Testklasse mit voneinander abhängigen Testfällen würde nach mehrmaligem Ausführen fehlschlagen.

Darüber hinaus werden von JUnit 5 automatisch **nach jedem Testfall alle Attribute der Testklasse zurückgesetzt**.

Für manche Klassen, z. B. eine Collection, kann es dennoch sinnvoll sein, diese in einer **@BeforeEach**-Methode zu initialisieren und mit Daten zu befüllen – siehe setUp().

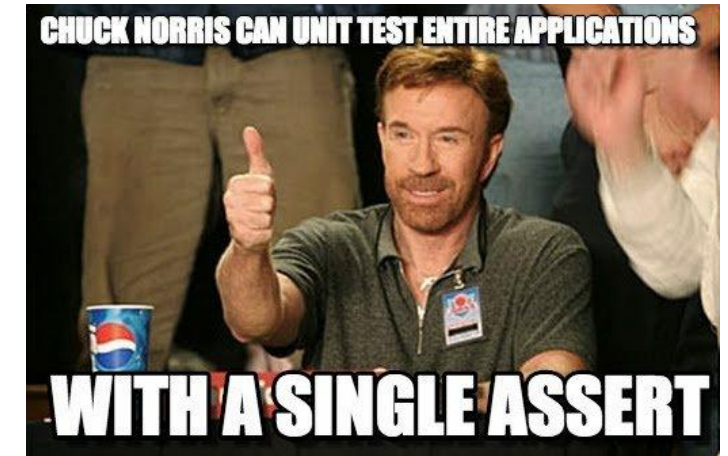
Ausgabe:

```
[Element 2]
```

```
[Element 2]
```


Best Practices: Wie viele Asserts?

Teste nur ein Konzept pro Testfall, d. h., halte die Anzahl an *asserts* pro Testfall möglichst gering! Die asserts sollten pro Testfall inhaltlich zusammengehören, z. B. erst die Anzahl der Elemente in einer Liste prüfen und anschließend die Werte der enthaltenen Elemente prüfen.



Das sind zwei unterschiedliche Konzepte bzw. Testfälle.

@Test

```
void twoPositiveNumbersAreCorrectlyAddedAndSubtracted() {
```

```
    // Arrange
```

```
    Calculator calculator = new Calculator();
```

```
    int numberA = 1;
```

```
    int numberB = 0;
```

```
    int expectedResult = 1;
```

```
    // Act
```

```
    int actualAddResult = calculator.add(numberA, numberB);
```

```
    int actualSubResult = calculator.subtract(numberA, numberB);
```

```
    // Assert
```

```
    assertEquals(expectedResult, actualAddResult);
```

```
    assertEquals(expectedResult, actualSubResult);
```

```
}
```

Teste die Subtraktion stattdessen in einem separaten Testfall:

@Test

```
void twoPositiveNumbersAreCorrectlySubtracted() {
```

```
    int expectedResult = 1;
```

```
    int actualSubResult = new Calculator().subtract(1, 0);
```

```
    assertEquals(expectedResult, actualSubResult);
```

```
}
```

Best Practices: Gleitkommawerte 1/2

Achtung beim Vergleich von float- und double-Werten:

```
double a = 0.1 + 0.2; // 0.30000000000000004  
double b = 0.3;      // 0.3
```

```
if(a == b) { // Funktioniert nicht immer. 😬  
    ...  
}
```

// oder

```
assertEquals(a, b); // Funktioniert nicht immer. 😬
```

Best Practices: Gleitkommawerte 2/2

Das Problem:

Wenn Gleitkommazahlen (float oder double) berechnet werden, kann es passieren, dass kleinere Rundungsfehler auftreten. Dies führt dazu, dass zwei Werte, die beinahe gleich sind, nicht als gleich evaluiert werden (z. B. bei `==`, `equals` und `assertEquals`).

Lösung:

- Wenn möglich, die Einheit reduzieren und stattdessen long verwenden. Beispiel: 123 Cent statt 1,23 Euro.
- Einen Toleranzwert definieren, der angibt, um wie viel sich die Werte unterscheiden dürfen.

Beispiel:

```
if(Math.abs(a - b) <= 0.0000000000000001) { ... } // Vergleich mit Delta bzw. Toleranz.
```

// oder

```
assertEquals(a, b, 0.0000000000000001); // Der dritte Parameter ist erlaubtes Delta bzw. Toleranz.
```

Best Practices: Clean Code

Schreibe verständliche Tests. Schreibe **Hilfsmethoden**, um das Testen zu vereinfachen, um Tests verständlicher zu machen und um Redundanz zu vermeiden. #refactoring

@Test

```
public void testGetPageHierarchyHasRightTags() {
    WikiPage pageOne = crawler.addPage(root, PathParser.parse("PageOne"));
    crawler.addPage(root, PathParser.parse("PageOne.ChildOne"));
    crawler.addPage(root, PathParser.parse("PageTwo"));

    request.setResource("root");
    request.addInput("type", "pages");
    Responder responder = new SerializedPageResponder();
    SimpleResponse response = (SimpleResponse)
        responder.makeResponse(new FitNesseContext(root), request);
    String xml = response.getContent();

    assertEquals("text/xml", response.getContentType());
    assertSubString("<name>PageOne</name>", xml);
    assertSubString("<name>PageTwo</name>", xml);
    assertSubString("<name>ChildOne</name>", xml);
}
```



@Test

```
public void testGetPageHierarchyHasRightTags() {
    givenPages("PageOne", "PageOne.ChildOne", "PageTwo");

    whenRequestIsIssued("root", "type:pages");

    thenResponseShouldContain(
        "<name>PageOne</name>",
        "<name>PageTwo</name>",
        "<name>ChildOne</name>"
    );
}

private void givenPages(List<String> pages) { ... } // Hilfsmethoden

private void whenRequestIsIssued(String root, String type) { ... }

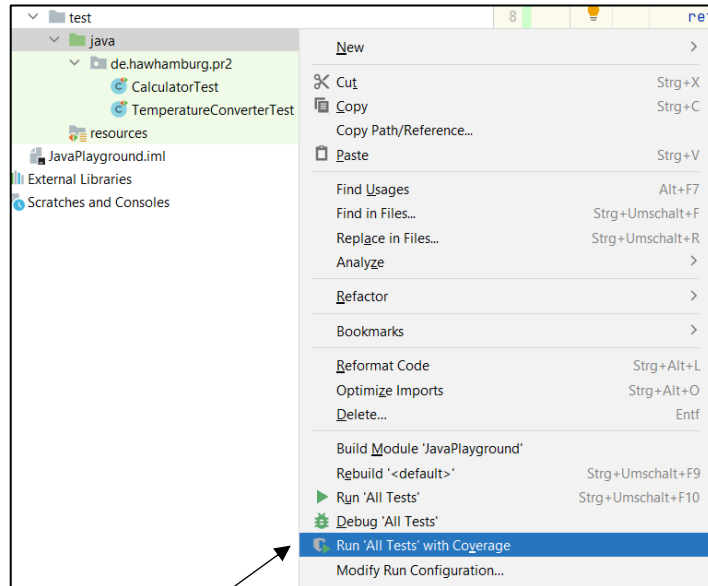
private void thenResponseShouldContain(List<String> responses) { ... }
```



Best Practices: Test Coverage 1/4

aka „Testabdeckung“

2. Test Coverage Ergebnisse.



1. Test Coverage starten.

A screenshot of an IDE showing the source code of a `TemperatureConverter` class. The code is as follows:

```
1 package de.hawhamburg.pr2;
2
3 import java.security.InvalidParameterException;
4
5 public class TemperatureConverter {
6     public double convert(double value, String convertToUnit) {
7         if ("Fahrenheit".equals(convertToUnit)) {
8             return (value * (9d / 5d)) + 32;
9         } else if ("Celsius".equals(convertToUnit)) {
10            return (value - 32) * (5d / 9d);
11        } else {
12            throw new InvalidParameterException("Temperaturumwandlung ist nicht möglich.");
13        }
14    }
15 }
```

Below the code, a table shows the test coverage results:

Element	Class, %	Method, %	Line, %
de.hawhamburg.pr2	100% (2/2)	100% (2/2)	62% (5/8)
Calculator	100% (1/1)	100% (1/1)	100% (2/2)
TemperatureConverter	100% (1/1)	100% (1/1)	50% (3/6)

A red box highlights the `if` and `else if` conditions in the `convert` method. A red arrow points from this box to a red text box on the right.

**Nicht alle Bedingungen der if-Anweisung werden getestet!
Die Testabdeckung sollte möglichst hoch sein.**

```
class TemperatureConverterTest {
    @Test
    void givenCelsius_whenConvert_thenReturnFahrenheit() {
        var temperatureConverter = new TemperatureConverter();
        String conversionOption = "Fahrenheit";
        var celsius = 10;
        var expectedFahrenheit = 50;

        var actualFahrenheit = temperatureConverter.convert(celsius, conversionOption);

        assertEquals(expectedFahrenheit, actualFahrenheit);
    }
}
```

Wie hoch???
Darüber wird
gestritten...



Best Practices: Test Coverage 2/4

1 **Line Coverage:**

2
3 Misst den Anteil der Codezeilen, die während der Ausführung der Tests
4 mindestens einmal ausgeführt werden.
5

Branch Coverage:

Misst den Anteil der Verzweigungen (z. B. in if-else-Anweisungen, Anweisungen), die während der Tests alle möglichen Ausgänge haben.



→ Branch Coverage ist aufwendiger, sollte aber präferiert werden.

Best Practices: Test Coverage 3/4

```
boolean isOfLegalAge(Integer age) {  
    if (age != null && age >= 18) {  
        return true;  
    }  
    return false;  
}
```

100% Line Coverage:

Alle Zeilen werden abgedeckt.

D. h. jeweils ein Testfall für `isOfLegalAge(17)` und `isOfLegalAge(18)`.

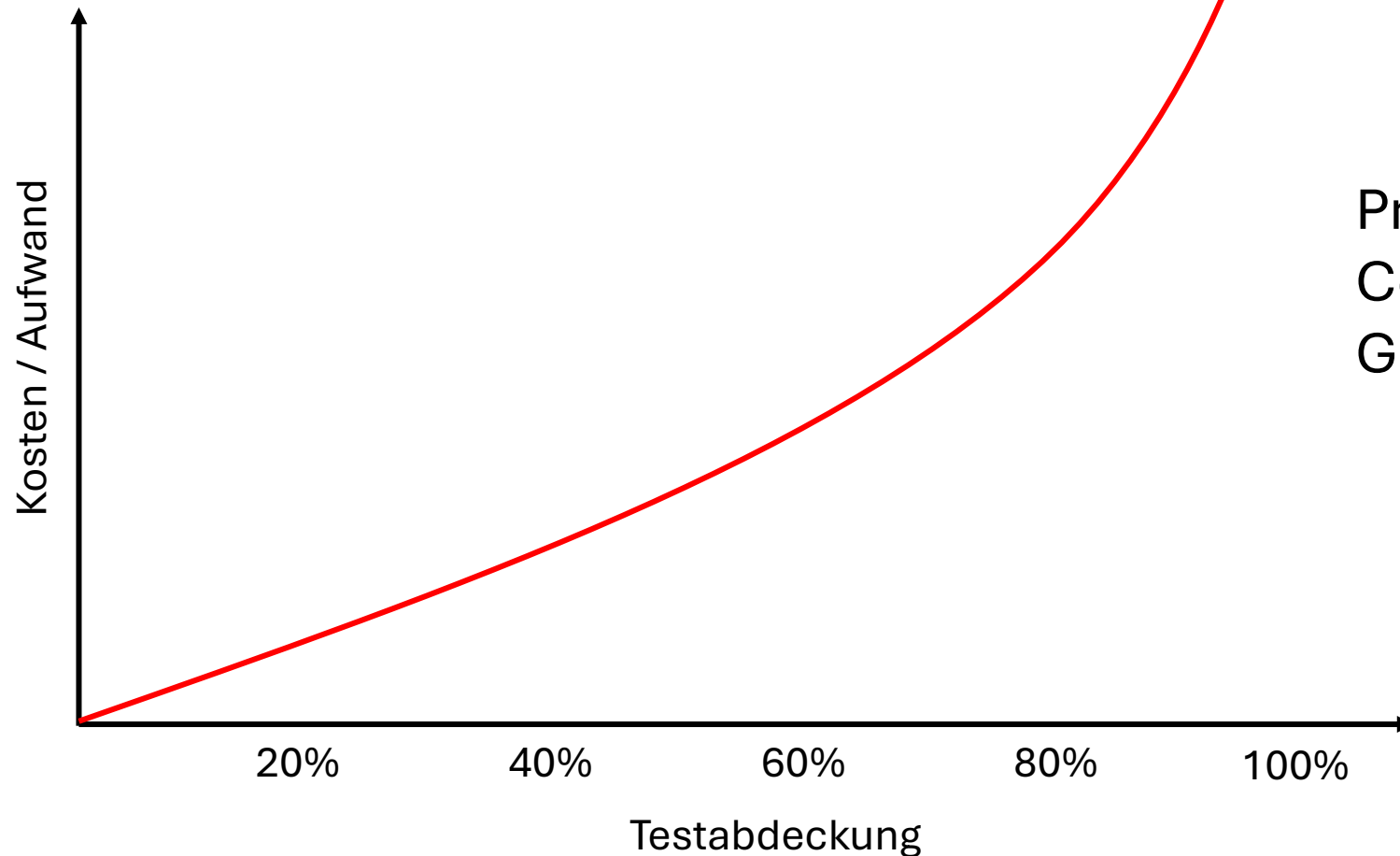
100% Branch Coverage:

Alle möglichen Pfade werden abgedeckt.

D. h. jeweils ein Testfall für `isOfLegalAge(17)`, `isOfLegalAge(18)` und `isOfLegalAge(null)`.



Best Practices: Testcoverage 4/4



Priorisiere Tests für kritischen Code anstatt z. B. generierte Getter/Setter zu testen.



Exkurs: Mutation-Tests

Mutation-Tests verändern (mutieren) den Code, um Fälle zu finden, die nicht durch Testfälle abgedeckt werden. Zum Beispiel ersetzen sie Bedingungen durch Konstanten (true/false) oder lassen Funktionsaufrufe weg.

Es gibt Tools, um Mutationen automatisiert durchzuführen wie zum Beispiel <https://pitest.org/>

"To put it another way - PIT runs your unit tests against automatically modified versions of your application code. When the application code changes, it should produce different results and cause the unit tests to fail. If a unit test does not fail in this situation, it may indicate an issue with the test suite."

```
19
20     List<Student> findByUniversity(String universityName) {
21         universityName = universityName.trim();
22
23         List<Student> result = new ArrayList<>();
24         for (Student student : students) {
25             if (student.university().contains(universityName)) {
26                 result.add(student);
27             }
28         }
29         return result;
30     }
31 }
```

Mutations

```
1. removed call to java/util/List::of → KILLED
2. Removed assignment to member variable students → KILLED
3. removed call to de/hawhamburg/Student::<init> → KILLED
11 4. removed call to de/hawhamburg/Student::<init> → KILLED
5. removed call to de/hawhamburg/Student::<init> → KILLED
6. removed call to de/hawhamburg/Student::<init> → KILLED
7. removed call to de/hawhamburg/Student::<init> → KILLED
21 1. replaced call to java/lang/String::trim with receiver → SURVIVED Covering tests
2. removed call to java/lang/String::trim → KILLED
23 1. removed call to java/util/ArrayList::<init> → KILLED
```

Best Practices: Edge Cases



Randfälle bzw. „Edge Cases“ sind Werte oder Eingaben, die am Rande des zulässigen Bereichs liegen oder außergewöhnliche Umstände widerspiegeln, z. B.:

- **null, 0** und **negative Werte**.
- **Leere Listen**, Listen mit nur einem Element, Listen mit vielen Elementen.
- **Leere Eingaben** (z. B. leerer String) oder Eingaben mit **minimaler** Länge (z. B. `""`).
- **Lange Eingaben** oder Eingaben mit **maximaler** Länge.
- **Sonderzeichen** oder ungültige Zeichen.
- **Falscher Datentyp** bzw. unerwartete Formate oder Strukturen.
- **Zeitberechnungen** wie Schaltjahre oder Sommer-/Winterzeit.
- Mathematische Berechnungen wie **Division durch 0**.
- **Überlauf** von Zahlen, z. B.: $(2147483647 + 1) \rightarrow -2147483648$.
- **Exceptions**.

Best Practices: Edge Cases – Exceptions

Mit *assertThrows* lässt sich prüfen, ob eine bestimmte Exception geworfen wird:

```
class PlayerTest {  
  
    @Test  
    void whenPlayerIsAddedToNonExistingGameThrowException() {  
        long invalidGameId = -1;  
        assertThrows(GameNotExistException.class, () -> player.addToGame(invalidGameId));  
    }  
}
```

Best Practices: Äquivalenzklassen

Äquivalenzklassen in Unit-Tests sind **Gruppen von Eingabewerten**, die im Hinblick auf das erwartete Verhalten der getesteten Funktion **als gleichwertig betrachtet** werden. Anstatt jeden einzelnen möglichen Wert zu testen, wählt man **repräsentative Werte aus jeder Äquivalenzklasse** aus. Das reduziert den Testaufwand, während die Testabdeckung hoch bleibt.

```
class EquivalenceDemo {  
  
    String isAllowedToDrive(Person person) {  
        if(isOfLegalAge(person)) {  
            return person.name() + " darf Auto fahren.";  
        } else {  
            return person.name() + " darf kein Auto fahren.";  
        }  
    }  
  
    private static boolean isOfLegalAge(Person person) {  
        return person.age() >= 18;  
    }  
}
```

Äquivalenzklassen:

Klasse 1: $\leq 17 \rightarrow$ "... darf kein Auto Fahren."

Klasse 2: $\geq 18 \rightarrow$ "... darf Auto fahren."

```
class EquivalenceClassTest {  
  
    @Test  
    void isAboveLegalAge() {  
        Person sara = new Person("Sara", 18);  
        String actual = new EquivalenceClassDemo().isAllowedToDrive(sara);  
        assertEquals("Sara darf Auto fahren.", actual);  
    }  
  
    @Test  
    void isAboveLegalAge2() {  
        Person sara = new Person("Sara", 19);  
        String actual = new EquivalenceClassDemo().isAllowedToDrive(sara);  
        assertEquals("Sara darf Auto fahren.", actual);  
    }  
  
    @Test  
    void isBelowLegalAge() {  
        Person sara = new Person("Sara", 17);  
        String actual = new EquivalenceClassDemo().isAllowedToDrive(sara);  
        assertEquals("Sara darf kein Auto fahren.", actual);  
    }  
  
    @Test  
    void isBelowLegalAge2() {  
        Person sara = new Person("Sara", 16);  
        String actual = new EquivalenceClassDemo().isAllowedToDrive(sara);  
        assertEquals("Sara darf kein Auto fahren.", actual);  
    }  
}
```

Unnötige Testfälle, da selbe Äquivalenzklasse wie vorheriger Testfall.

Diese Testfälle erhöhen nicht die Qualität des Tests, da sie nicht wirklich etwas neues Testen.

Folgeproblem: bei Änderungen in der Klasse EquivalenceDemo müssten auch mehr Testfälle korrigiert werden, was den Entwicklungsaufwand unnötig erhöht.

Best Practices: JUnit – Dialekte

In der Praxis werden häufig Erweiterungen für JUnit eingesetzt.

Beispiele:

- [AssertJ](#) / [Truth](#)
- [Hamcrest](#)

Diese bieten u. a. weitere assert-Methoden, verständlichere Fehlermeldungen und alternative Schreibweisen, die Tests lesbarer machen.

```
import static org.assertj.core.api.Assertions.*;

// basic assertions
assertThat(frodo.getName()).isEqualTo("Frodo");
assertThat(frodo).isNotEqualTo(sauron);

// chaining string specific assertions
assertThat(frodo.getName()).startsWith("Fro")
    .endsWith("do")
    .isEqualToIgnoringCase("frodo");
```

```
import static com.google.common.truth.Truth.*;

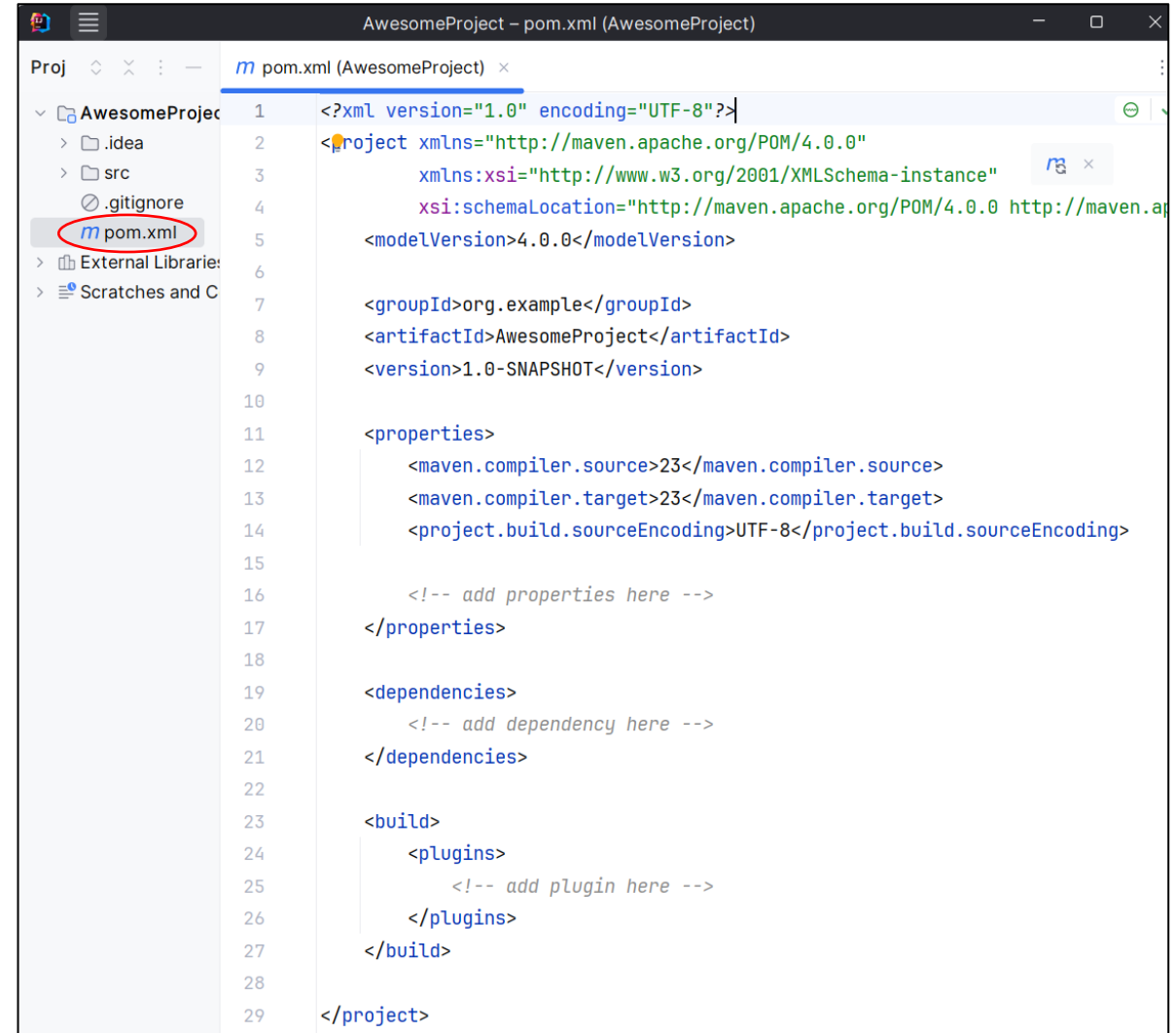
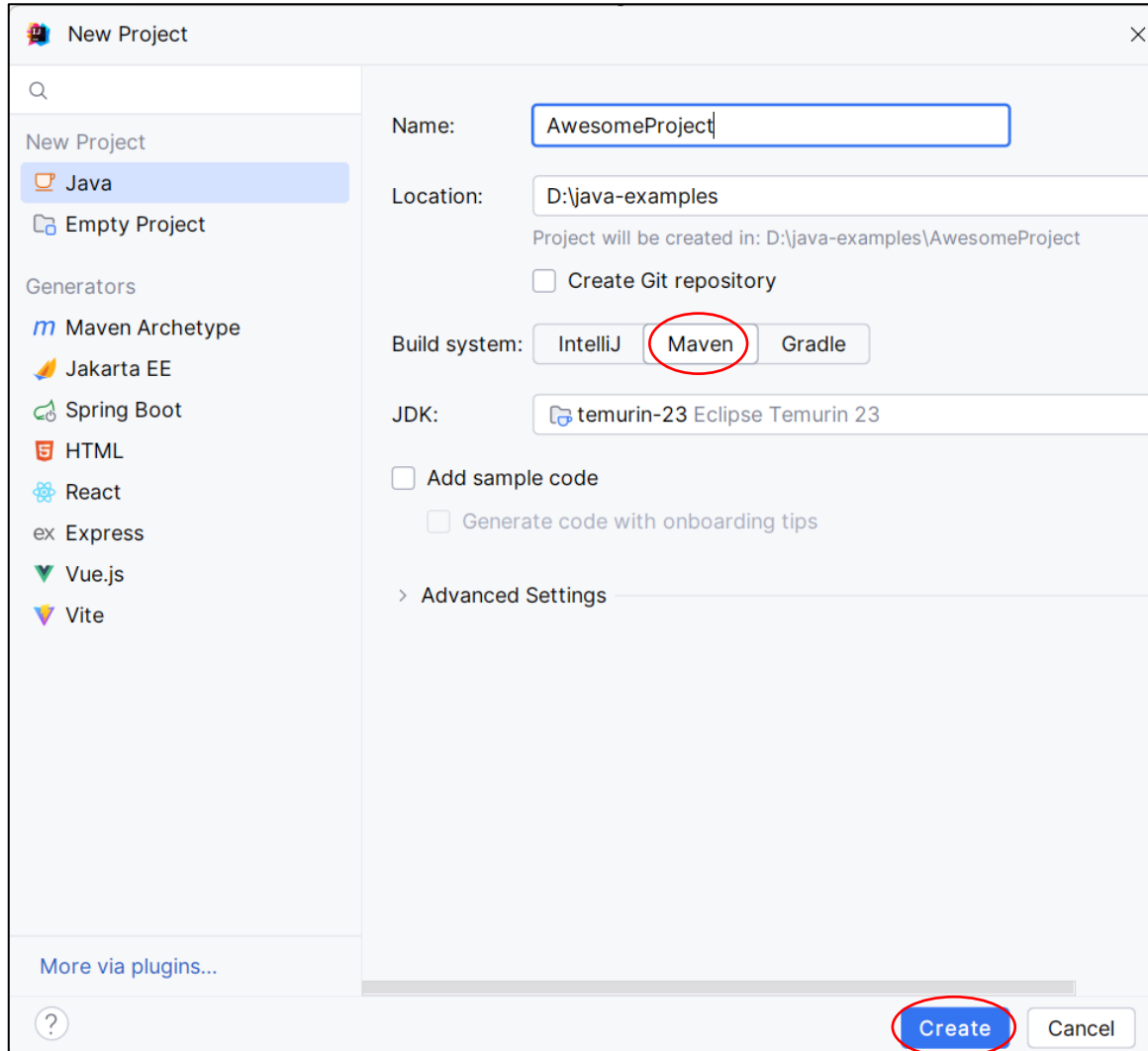
String string = "awesome";
assertThat(string).startsWith("awe");

Iterable<Color> googleColors = googleLogo.getColors();
assertThat(googleColors)
    .containsExactly(BLUE, RED, YELLOW, BLUE, GREEN, RED)
    .inOrder();
```

```
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

Biscuit theBiscuit = new Biscuit("Ginger");
Biscuit myBiscuit = new Biscuit("Ginger");
assertThat(theBiscuit, equalTo(myBiscuit));
```

JUnit mit Maven 1/2



JUnit mit Maven 2/2

```
<properties>
  <!-- Java settings -->
  <maven.compiler.source>25</maven.compiler.source>
  <maven.compiler.target>25</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <!-- Shared version properties -->
  <pitest-version>1.19.1</pitest-version>
</properties>

<dependencies>
  <!-- JUnit 5 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.12.2</version>
    <scope>test</scope>
  </dependency>
  <!-- AssertJ JUnit extension that adds assertThat (alternative: Truth) -->
  <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>assertj-core</artifactId>
    <version>3.27.3</version>
    <scope>test</scope>
  </dependency>
  <!-- Truth JUnit extension that adds assertThat (alternative: AssertJ) -->
  <dependency>
    <groupId>com.google.truth</groupId>
    <artifactId>truth</artifactId>
    <version>1.4.4</version>
    <scope>test</scope>
  </dependency>
  <!-- PIT mutation tests. -->
  <dependency>
    <groupId>org.pitest</groupId>
    <artifactId>pitest</artifactId>
    <version>${pitest-version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.pitest</groupId>
    <artifactId>pitest-junit5-plugin</artifactId>
    <version>1.2.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```
<build>
  <plugins>
    <!-- pitest code mutations during unit tests -->
    <!-- To start a mutation test with all mutators, run the following command with by double pressing CTRL: -->
    <!-- mvn test-compile org.pitest:pitest-maven:mutationCoverage -->
    <plugin>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-maven</artifactId>
      <version>${pitest-version}</version>
      <configuration>
        <!-- Define which packages will be mutated / used for pitest. -->
        <targetClasses>
          <param>de.hawhamburg*</param>
        </targetClasses>
        <targetTests>
          <param>de.hawhamburg*</param>
        </targetTests>
        <mutators>
          <!-- Activate all available mutators. -->
          <mutator>ALL</mutator>
        </mutators>
      </configuration>
    </plugin>
  </plugins>
</build>
```



Welche Versionen gerade aktuell ist, findest du z. B. hier heraus:
<https://mvnrepository.com/repos/central>

Buchempfehlungen

- Clean Craftmanship (Robert C. Martin, ISBN: 978-0-13-691571-3)
- Test-Driven Development (Kent Beck, ISBN: 978-0321146533)

